

ITM SLS Baroda University

FACULTY LABORATORY MANUAL

For

III Semester CSE

Session: 2025-2026

Object Oriented Programming using Java

Subject Code: C2311C1



"Think Big... Think Beyond"

Prepared by:

Ms. Diksha Durgapal

Assistant Professor

School of Computer Science Engineering & Technology

June -2025

Institute of Technology & Management Universe

*Dhanora Tank Road, Paldi Village, Halol Highway,
Near Jarod, Vadodara Dist, Gujarat, INDIA-391510*

(Approved by AICTE, New Delhi, and Affiliated to Gujarat Technological University, Ahmadabad)



BARODA UNIVERSITY

Vadodara | Gujarat | India

"Think Big... Think Beyond"

STUDENT CERTIFICATE

*This is to certify Mr.\ Miss student of
Btech in Computer Science & Engineering, Enrollment No.
..... has satisfactorily completed his/her term-work
..... in Object Oriented Programming using Java for term
ending in.....2025.*

Date:

Place:

Signature of Faculty

(.....)

Date

Signature of HOD

(.....)

Date

1. Lab Syllabus

ITM SLS BARODA UNIVERSITY
B.Tech SEMESTER: III
COMPUTER SCIENCE & ENGINEERING

Subject Name: Object Oriented Programming using Java

Subject Code: C2310C1

Evaluation Scheme

Teaching Scheme			Credits	Examination Marks				Total Marks
L	T	P	C	Theory Marks		Practical marks		
				External	Internal	External	Internal	
3	0	4	5	40	60	0	50	150

List of Experiments:

- Write a Program that displays Welcome to Java, Learning Java Now and Programming is fun.
- Write a program that solves the following equation and displays the value x and y:
1) $3.4x + 50.2y = 44.5$ 2) $2.1x + .55y = 5.9$
Assume Cramer's rule to solve equation:
 $ax + by = e$ $x = \frac{ed - bf}{ad - bc}$ $cx + dy = f$ $y = \frac{af - ec}{ad - bc}$
- Write a program that reads a number in meters, converts it to feet, and displays the result.
- Body Mass Index (BMI) is a measure of health on weight. It can be calculated by taking your weight in kilograms and dividing by the square of your height in meters. Write a program that prompts the user to enter a weight in pounds and height in inches and displays the BMI.
Note:- 1 pound=.45359237 Kg and 1 inch=.0254 meters.
- Write a program that prompts the user to enter three integers and display the integers in decreasing order.
- Write a program that prompts the user to enter a letter and check whether a letter is a vowel or constant.
- Assume a vehicle plate number consists of three uppercase letters followed by four digits. Write a program to generate a plate number.
- Write a program that reads an integer and displays all its smallest factors in increasing order. For example, if input number is 120, the output should be as follows: 2,2,2,3,5.
- Write a method with following method header: `public static int gcd(int num1, int num2)`
Write a program that prompts the user to enter two integers and compute the gcd of two integers.
- Write a test program that prompts the user to enter ten numbers, invoke a method to reverse the numbers, display the numbers.
- Write a program that generate 6*6 two-dimensional matrix, filled with 0's and 1's , display the matrix, check every row and column have an odd number's of 1's.
- Write a program that creates a Random object with seed 1000 and displays the first 100 random integers between 1 and 49 using the NextInt (49) method.
- Write a program to implement Inheritance & its types
- Write a program to implement Exception handling
- Write a Program to implement multithreaded program
- Write a GUI program that use button to move the message to the left and right and use the radio button to change the colour for the message displayed
- Write a Program to implement Class Modelling

ACTIVE LEARNING ASSIGNMENTS: Students will be grouped in teams of a maximum of 3. Each group will select one mini-project from the options provided by the subject teacher and prepare it based on their understanding of the subject.

List of mini-projects:

1. Feedback form
2. Favorite programming language polling
3. Media Player Mini
4. Simple Calculator
5. To-do list
6. Quiz app
7. Mini Survey form

2. List of Experiment

S. No.	Date of Experiment	Title	Page No.	Grade	Signature
1		Write a Program that displays Welcome to Java, Learning Java Now and Programming is fun			
2		Write a program that solves the following equation and displays the value x and y: 1) $3.4x + 50.2y = 44.5$ 2) $2.1x + .55y = 5.9$ Assume Cramer's rule to solve equation $ax + by = e$ $x = (ed - bf) / (ad - bc)$ $cx + dy = f$ $y = (af - ec) / (ad - bc)$			
3		Write a program that reads a number in meters, converts it to feet, and displays the result			
4		Write a program that prompts the user to enter a weight in pounds and height in inches and displays the BMI.			
5		Write a program that prompts the user to enter three integers and display the integers in decreasing order.			
6		Write a program that prompts the user to enter a letter and check whether a letter is a vowel or constant.			
7		Write a program to generate a plate number			
8		Write a program that reads an integer and displays all its smallest factors in increasing order.			
9		Write a program that prompts the user to enter two integers and compute the gcd of two integers.			
10		Write a test program that prompts the user to enter ten numbers, invoke a method to reverse the numbers, display the numbers			
11		Write a program that generate 6*6 two-dimensional matrix, filled with 0's and 1's , display the matrix, check every row and column have an odd number's of 1's.			
12		Write a program that creates a Random object with seed 1000 and displays the first 100 random integers between 1 and 49 using the NextInt (49) method.			
13		Write a program to implement Inheritance & its types			
14		Write a program to implement Exception			

		handling			
15		Write a Program to implement multithreaded program			
16		Write a GUI program that use button to move the message to the left and right and use the radio button to change the colour for the message Displayed			
17		Write a Program to implement Class Modeling			

3. Lab Plan

S. No	List of Experiment	Dates	
		Performed Date	Back Log Date
1	Write a Program that displays Welcome to Java, Learning Java Now and Programming is fun		
2	Write a program that solves the following equation and displays the value x and y: 1) $3.4x + 50.2y = 44.5$ 2) $2.1x + .55y = 5.9$ Assume Cramer's rule to solve equation $ax + by = e$ $x = (ed - bf) / (ad - bc)$ $cx + dy = f$ $y = (af - ec) / (ad - bc)$		
3	Write a program that reads a number in meters, converts it to feet, and displays the result		
4	Write a program that prompts the user to enter a weight in pounds and height in inches and displays the BMI.		
5	Write a program that prompts the user to enter three integers and display the integers in decreasing order.		
6	Write a program that prompts the user to enter a letter and check whether a letter is a vowel or constant.		
7	Write a program to generate a plate number		
8	Write a program that reads an integer and displays all its smallest factors in increasing order.		
9	Write a program that prompts the user to enter two integers and compute the gcd of two integers.		
10	Write a test program that prompts the user to enter ten numbers, invoke a method to reverse the numbers, display the numbers		
11	Write a program that generate 6*6 two-dimensional matrix, filled with 0's and 1's , display the matrix, check every row and column have an odd number's of 1's.		

12	Write a program that creates a Random object with seed 1000 and displays the first 100 random integers between 1 and 49 using the NextInt (49) method.		
13	Write a program to implement Inheritance & its types		
14	Write a program to implement Exception handling		
15	Write a Program to implement multithreaded program		
16	Write a GUI program that use button to move the message to the left and right and use the radio button to change the colour for the message displayed		
17	Write a Program to implement Class Modeling		

Practical No: 1

Aim: Write a Program that displays "Welcome to Java", "Learning Java Now" and "Programming is fun".

Theory: This program demonstrates how to use the `System.out.println()` function in Java to display multiple lines of text. It is often the first program written when learning a new language and helps understand the basic syntax of Java.

The basic syntax of java program is:

```
public class classname {  
    public static void main(String args[ ]){  
        System.out.println("Content");  
    }  
}
```

public: Public is an access modifiers which tells that program is accessible by everyone.

class: Keyword used to declare a class.

Classname: name of the class. It must match with the file name. For eg. If file name is `Hello.java` then class name must be **Hello**

public static void main (String args[]): This is the main method of the java program, also called as the entry point of the java program. **Public** means that the program is accessible by JVM from anywhere. **Static** means JVM does not need to create an object to call this method. **void** means this method does not return any value. **String args[]** means that program accepts command-line arguments in the form of a string array.

System.out.println("content"); : We use this to print the output on the screen. **System** is a built-in Java class. **out** is a static member of the System class & is the standard output stream. **println** is a method that prints the message to the console and moves the cursor to the next line.

Source Code:

```
public class WelcomeMessage {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java");  
        System.out.println("Learning Java Now");  
        System.out.println("Programming is fun");  
    }  
}
```

Output:

```
Welcome to Java  
Learning Java Now  
Programming is fun
```

Conclusion: This program gives the better understanding and demonstration of the java language syntax.

Question-Answers:

1. Which method is used to print text in Java?

`System.out.println()`

2. Can you print multiple lines using one print statement?

Yes, by using `\n` for new lines.

3. Why is the `main()` method static in Java?

The `main()` method is static so it can be called without creating an object of the class. The JVM needs to call this method to start the program.

4. What are the rules for naming a class in Java?

- Class names must start with a letter.
- Can include letters, digits, underscore `_`, and dollar sign `$`.
- Should start with an uppercase letter by convention.
- Cannot use Java reserved keywords.

5. What is the use of `System.out.println()`?

It is used to print output to the console. `System` is a class, `out` is a static object of `PrintStream`, and `println()` is a method to print a line.

Practical No: 2

Aim: Write a program that solves the following equation and displays the value x and y: 1) $3.4x + 50.2y = 44.5$ 2) $2.1x + .55y = 5.9$

Assume Cramer's rule to solve equation:

$$ax + by = e \quad x = \frac{ed - bf}{ad - bc} \quad cx + dy = f \quad y = \frac{af - ec}{ad - bc}$$

Theory: A variable in Java is a container used to store data values during program execution. Every variable must be declared with a data type.

Types of Variables:

1. **Local Variables** – declared inside a method.
2. **Instance Variables** – declared inside a class, but outside methods.
3. **Static Variables** – declared using static keyword; shared across all instances.

Data types in Java specify the type of data a variable can hold. Java is a **statically typed** language, which means all variables must be declared with a data type before use.

Types of Data-types in java:

1. Primitive datatype: byte, short, int, long, float, double, char, boolean
2. Non-Primitive datatypes: string, arrays, classes, interface

Source Code:

```
public class CramerRule {  
    public static void main(String[] args) {  
        double a = 3.4, b = 50.2, e = 44.5;  
        double c = 2.1, d = 0.55, f = 5.9;  
  
        double denominator = a * d - b * c;  
        double x = (e * d - b * f) / denominator;  
        double y = (a * f - e * c) / denominator;  
  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
    }  
}
```

Output:

```
x = 2.623901496861419  
y = 0.7087397392563978
```

Conclusion: from this practical we learnt how to declare the variables, what are the various datatypes and how to use primitive datatypes

Question-Answers:

1. **What are data types in Java?**
Data types define the kind of data a variable can store. Java supports primitive and non-primitive data types.
2. **What is the difference between primitive and non-primitive data types?**

Primitive	Non-Primitive
Store simple values	Store objects/references
Fast and memory efficient	More flexible and powerful
Examples: int, float	Examples: String, Array

3. What is the size of char in Java and why?

char takes 2 bytes because Java uses Unicode, which supports international character sets.

Practical No: 3

Aim: Write a program that reads a number in meters, converts it to feet, and displays the result.

Theory: This program demonstrates type conversion (from String input to double) and the use of user input through the Scanner class. It also includes basic arithmetic operations and the standard output mechanism.

Type Conversion in Java is the process of converting one data type into another. Java supports two types of type conversion: implicit & explicit type conversion

Rules for Type Conversion Order: byte → short → int → long → float → double

The **Scanner** class is used to **take input** from the user in Java. It is found in the java.util package. Methods used for reading input from users using scanner class are: nextFloat(), nextInt(), nextDouble(), nextLine(), next()

Using type conversion, Java allows flexible data manipulation between types. The Scanner class makes it easy to accept user input, making programs more interactive.

Source Code:

```
import java.util.Scanner;

public class MeterToFeet {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter value in meters: ");
        double meters = input.nextDouble();
        double feet = meters * 3.28084;
        System.out.println(meters + " meters = " + feet + " feet");
    }
}
```

Output:

```
Enter value in meters: 10
10.0 meters = 32.8084 feet
```

Conclusion: Better understanding of using scanner class as well type conversions & arithmetic operations.

Question-Answers:

1. What is type conversion in Java?

Type conversion in Java is the process of converting a variable from one data type to another. It can be either:

Implicit (Widening) – done automatically.

Explicit (Narrowing) – done manually using type casting.

2. What is the Scanner class and what is its purpose?

The Scanner class is used to read input from the user via keyboard. It is a part of the java.util package and simplifies input handling.

3. How do you import and use the Scanner class in Java?

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
int x = sc.nextInt(); // input an integer
```

4. What happens if we input a float into nextInt() method?

A java.util.InputMismatchException will be thrown because nextInt() expects an integer, not a decimal.

5. What is data loss in type casting?

When converting from a larger type to a smaller type (e.g., double to int), the fractional part is removed, which leads to data loss.

6. Can we type cast boolean in Java?

No, Java does not allow type casting for boolean to/from other types. boolean is not compatible with numeric or character types.

Practical No: 4

Aim: Body Mass Index (BMI) is a measure of health on weight. It can be calculated by taking your weight in kilograms and dividing by the square of your height in meters. Write a program that prompts the user to enter a weight in pounds and height in inches and displays the BMI. Note:- 1 pound=.45359237 Kg and 1 inch=.0254 meters.

Theory: The program uses the **Scanner** class from the java.util package to allow the user to enter weight and height at runtime.

Scanner is a simple yet powerful way to handle **interactive input** from the console. In this program, it helps make the application dynamic and user-friendly.

Source Code:

```
import java.util.Scanner;

public class BMICalculator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter weight in pounds: ");
        double weight = input.nextDouble();

        System.out.print("Enter height in inches: ");
        double height = input.nextDouble();

        double weightInKg = weight * 0.45359237;
        double heightInMeters = height * 0.0254;

        double bmi = weightInKg / (heightInMeters * heightInMeters);

        System.out.println("BMI is: " + bmi);
    }
}
```

Output:

```
Enter weight in pounds: 150
Enter height in inches: 70
BMI is: 21.52253815221916
```

Conclusion:

This program showcases a real-world health application of Java and reinforces the integration of input/output handling, data conversion, and numeric calculations within a clean, modular code structure.

Question-Answers:

- 1. What Java class is used to take input?**
Scanner
- 2. Which data type is used to store the user input and why?**
The double data type is used to store inputs and calculations because BMI involves decimal values and floating-point division, which require precision.

3. What happens if we use int instead of double in this program?

Using int would lead to loss of decimal precision, which could produce incorrect or rounded BMI results. double ensures accurate calculation.

4. Explain how Scanner improves user interaction.

Scanner allows the user to enter inputs at runtime, making the program dynamic, interactive, and more useful for real-world applications like health apps.

Practical No: 5

Aim: Write a program that prompts the user to enter three integers and display the integers in decreasing order.

Theory: This program demonstrates conditional statements, comparison operators, and the use of if-else logic to determine order among values.

Conditional statements in Java allow the program to make decisions and execute code blocks based on true or false conditions. These are essential for controlling program flow.

Types of conditional statements are: if, if-else, if-elseif ladder, switch statement

Comparison(relational) operators are used to compare two values or variables. These comparisons return either true or false, and are mostly used in conditional expressions.

Types of comparison operators are: ==, !=, >, <, <=, >=

Source Code:

```
import java.util.Scanner;

public class SortThreeNumbers {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter three integers: ");
        int a = input.nextInt();
        int b = input.nextInt();
        int c = input.nextInt();

        if (a >= b && a >= c) {
            if (b >= c)
                System.out.println(a + " " + b + " " + c);
            else
                System.out.println(a + " " + c + " " + b);
        } else if (b >= a && b >= c) {
            if (a >= c)
                System.out.println(b + " " + a + " " + c);
            else
                System.out.println(b + " " + c + " " + a);
        } else {
            if (a >= b)
                System.out.println(c + " " + a + " " + b);
            else
                System.out.println(c + " " + b + " " + a);
        }
    }
}
```

Output:

```
Enter three integers: 9 2 5
9 5 2
```

Question-Answers:

1. Why are comparison operators used in Java?

Comparison (relational) operators are used to **compare two values or expressions**, and return a boolean result (true or false). They are essential for forming conditions in if, while, or loop statements.

2. What is the difference between = and == in Java?

= is an **assignment** operator (used to assign a value).

== is a **comparison** operator (used to compare two values).

3. What is the purpose of an if-else statement?

An if-else statement lets you choose between two paths: one when the condition is true, and the other when it's false.

4. Can switch be used with strings in Java?

Yes, since Java 7, the switch statement can be used with String values.

5. What is a switch statement, and when is it used?

The switch statement is used to perform **multi-way branching** based on the value of a single variable. It is preferred when you have **multiple fixed choices** like menu options or days of the week.

Practical No: 6

Aim: Write a program that prompts the user to enter a letter and check whether a letter is a vowel or constant

Theory: This program demonstrates character input, use of logical operators, and switch-case or if-else decision making.

Java does not have a direct nextChar() method in Scanner, so we read a string and extract the first character using:

```
Scanner sc = new Scanner(System.in);

char ch = sc.next().charAt(0); // takes the first character
```

This method is commonly used when reading single-character inputs like letters or menu options.

Source Code:

```
import java.util.Scanner;

public class VowelChecker {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a letter: ");
        char ch = input.next().charAt(0);

        if ("aeiouAEIOU".indexOf(ch) != -1)
            System.out.println(ch + " is a vowel.");
        else
            System.out.println(ch + " is a consonant.");
    }
}
```

Output:

```
Enter a letter: A
A is a vowel.
```

Question-Answers:

1. How do you read a single character from the user in Java?

By reading a string using next() and getting the first character using charAt(0):

```
char ch = sc.next().charAt(0);
```

2. What is the purpose of logical operators in Java?

Logical operators are used to combine multiple conditions in decision-making (if, while, etc.) and return true or false.

3. Explain the difference between && and ||.

- && (AND): True only if both conditions are true.
- || (OR): True if at least one condition is true.

4. When would you prefer switch-case over if-else?

Use switch-case when you are comparing a variable with multiple constant values, such as characters, integers, or strings. It makes the code more readable and efficient.

Practical No: 7

Aim: Assume a vehicle plate number consists of three uppercase letters followed by four digits. Write a program to generate a plate number.

Theory:

This program demonstrates the use of the Random class, type casting, and string manipulation. It introduces generating random values and combining characters and numbers to form a string. The Random class is used to generate pseudo-random numbers. It belongs to the java.util package and allows the creation of random values of various data types, such as int, double, float, long, and boolean.

Key Features of Random Class

- Pseudo-random: Numbers appear random but are generated algorithmically.
- Reproducible: With the same seed, the sequence of numbers can be reproduced.
- Versatile: Supports different data types.

Source Code:

```
import java.util.Random;

public class PlateGenerator {
    public static void main(String[] args) {
        Random rand = new Random();
        String plate = "";
        for (int i = 0; i < 3; i++) {
            plate += (char) ('A' + rand.nextInt(26));
        }
        for (int i = 0; i < 4; i++) {
            plate += rand.nextInt(10);
        }
        System.out.println("Generated Plate Number: " + plate);
    }
}
```

Output:

Generated Plate Number: XYZ1234

Conclusion:

The Random class is an essential utility in Java for generating unpredictable results in various applications like games, testing, and simulations. It supports multiple data types and offers flexibility in generating numbers within specific ranges. Understanding it helps in implementing dynamic and interactive Java programs.

Question-Answers:

1. What is the use of the Random class in Java?

The Random class is used to generate pseudo-random numbers of different data types such as int, float, double, and boolean.

2. Which package includes the Random class?

The Random class is part of the java.util package.

3. What is the difference between nextInt() and nextInt(n)?

- `nextInt()` returns a random integer from the entire `int` range.
- `nextInt(n)` returns a random integer between 0 (inclusive) and `n` (exclusive).

4. How can you generate a random number between 1000 and 9999 (4-digit)?

```
int number = rand.nextInt(9000) + 1000;
```

5. How to generate a random uppercase letter?

```
char letter = (char) (rand.nextInt(26) + 'A');
```

6. Can we generate random strings using the Random class?

Not directly. But by combining `nextInt()` and ASCII values or character arrays, we can generate random strings.

Practical No: 8

Aim: Write a program that reads an integer and displays all its smallest factors in increasing order. For example if input number is 120, the output should be as follows: 2,2,2,3,5.

Theory: This program makes use of loops, modulus operator, and conditional logic. It demonstrates iteration and how to perform divisibility checks.

Loops are control structures that allow code to be executed **repeatedly** based on a condition.

Types of loops: while, do-while & for loop

Here in this program, we have used for and while loop together. For loop iterates the number starting from 2(smallest prime number) till the number entered by the user whereas while loops continuously divides the iterated number to get the factor.

Source Code:

```
import java.util.Scanner;

public class SmallestFactors {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int number = input.nextInt();

        for (int i = 2; i <= number; i++) {
            while (number % i == 0) {
                System.out.print(i + " ");
                number = number / i;
            }
        }
    }
}
```

Output:

```
Enter an integer: 120
2 2 2 3 5
```

Conclusion:

This program demonstrates the use of loops, conditional statements, and the modulus operator to perform prime factorization. It efficiently reads user input, checks for divisibility, and prints all smallest factors in increasing order using Scanner, for, and while loops.

Question-Answers:

1. What is a loop in Java?

A loop is a control structure that allows code to execute repeatedly as long as a specified condition is true.

2. What is the difference between for and while loop?

- `for` loop is used when the number of iterations is known.
- `while` loop is used when the condition needs to be evaluated before each iteration and the count is unknown.

3. What is the use of do-while loop?

A `do-while` loop ensures that the block is executed at least once, as the condition is checked **after** the loop body.

4. Can loops be nested in Java?

Yes, one loop can be placed inside another, known as a nested loop, commonly used for pattern printing or matrices

Practical No: 9

Aim:

Write a method with following method header.

```
public static intgcd(int num1, int num2)
```

Write a program that prompts the user to enter two integers and compute the gcd of two integers.

Theory:

This Java uses the Scanner class to take input and a separate method to compute the GCD, demonstrating modular arithmetic, looping, and conditional logic.

Inside the method, a while loop repeatedly applies $a \% b$ until b becomes zero, at which point a holds the GCD. This approach shows the use of loops and methods for clean and reusable code structure.

Source Code:

```
import java.util.Scanner;

public class GCDCalculator {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter first integer: ");
        int a = input.nextInt();
        System.out.print("Enter second integer: ");
        int b = input.nextInt();

        while (a != b) {
            if (a > b)
                a -= b;
            else
                b -= a;
        }
        System.out.println("GCD is: " + a);
    }
}
```

Output:

```
Enter first integer: 24
Enter second integer: 36
GCD is: 12
```

Conclusion:

This program demonstrates the use of Java's core features like the Scanner class for input handling, while loop for iterative logic. It effectively uses Euclid's algorithm and modular arithmetic to calculate the GCD of two integers, showcasing Java's ability to perform structured, and efficient programming.

Question-Answers:

1. How does a while loop work in Java?

It repeatedly executes a block of code as long as the condition in the parentheses evaluates to true.

2. How do you display output to the console in Java?

Using `System.out.println()` for a line with a newline or `System.out.print()` for inline output.

Practical No: 10

Aim: Write a test program that prompts the user to enter ten numbers, invoke a method to reverse the numbers, display the numbers.

Theory:

This program uses arrays, loops, and methods. It also introduces array indexing and how to pass arrays to methods.

An array in Java is a data structure used to store multiple values of the same type in a single variable. It allows storing data in a contiguous block of memory, where each element is accessed using an index starting from 0. Arrays simplify operations like sorting, searching, and reversing by allowing indexed access and iteration through loops.

In this program, an array of size 10 is created to store the ten numbers input by the user.

A for loop is used to take 10 inputs from the user. Loops are essential for handling arrays efficiently, especially when dealing with repetitive input/output operations. This loop stores each input at the appropriate index in the array.

In Java, arrays are objects and can be passed to methods just like other objects. When an array is passed to a method, the method receives a reference to the original array (not a copy), so changes made to the array within the method affect the original array.

The reversing logic is placed inside a user-defined method for better modularity and reusability. The method swaps elements from both ends of the array, moving toward the center.

After the array is reversed, another loop is used to print the array values. Each element is accessed by its index and displayed using `System.out.println()` or `System.out.print()`.

Source Code:

```
import java.util.Scanner;

public class ReverseArray {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int[] numbers = new int[10];
        System.out.println("Enter 10 integers:");
        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = input.nextInt();
        }
        reverse(numbers);
        System.out.println("Reversed array:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
    }

    public static void reverse(int[] array) {
        int temp;
        for (int i = 0; i < array.length / 2; i++) {
            temp = array[i];
```

```

        array[i] = array[array.length - 1 - i];
        array[array.length - 1 - i] = temp;
    }
}
}

```

Output:

```

Enter 10 integers:
1 2 3 4 5 6 7 8 9 10
Reversed array:
10 9 8 7 6 5 4 3 2 1

```

Conclusion:

This program enhances understanding of array creation and manipulation, looping through arrays, defining and calling methods, passing arrays as arguments and working with references vs. values.

Question-Answers:

1. What data structure is used to store the input?

Array is used here for storing input. An array in Java is a collection of elements of the same data type stored in a contiguous memory location. Elements are accessed using an index, starting from 0.

2. Can arrays be passed to methods?

Yes, arrays can be passed as arguments to methods. They are passed **by reference**, so changes inside the method affect the original array..

3. What is a method in Java?

A method is a reusable block of code that performs a specific task. It helps in modular programming and avoids code repetition.

4. What does it mean to reverse an array?

Reversing an array means changing the order of elements so that the first becomes last, second becomes second-last, and so on.

5. What is the benefit of using methods with arrays?

It promotes code reuse, modularity, and better code organization, especially when performing tasks like searching, sorting, or reversing.

Practical No: 11

Aim: Write a program that generate 6*6 two-dimensional matrix, filled with 0's and 1's , display the matrix, check every row and column have an odd number's of 1's.

Theory:

This program uses 2D arrays, nested loops, and basic counting logic. It also practices generating random numbers and performing matrix-level traversal.

A **multidimensional array** in Java is essentially an array of arrays. The most common form is the **two-dimensional (2D) array**, which is used to store data in a table format — rows and columns. It is particularly useful when working with matrices, grids, or tabular data.

A 2D array is declared as: `int[][] matrix = new int[6][6];`

This statement creates a 6×6 matrix of integers. Each element is accessed using two indices: `matrix[i][j]`, where *i* represents the row index and *j* represents the column index. Java stores this internally as an array of 6 rows, with each row being an array of 6 columns.

Multidimensional arrays allow efficient traversal using **nested loops**, making them ideal for matrix operations, such as the one performed in this program.

To simulate binary values in the matrix, the program fills each cell with either 0 or 1 using Java's `Math.random()` function.

Matrix traversal is done using **nested for loops**, where:

- The outer loop iterates through each row.
- The inner loop iterates through each column within that row.

This structure is applied both for displaying the matrix and for validating the number of 1s in each row and column.

To validate whether each row and column has an odd number of 1s, the program counts the number of 1s using a counter variable inside a loop. After counting, it checks if the value is odd using the modulus operator `%`. This logic is applied separately for each row and each column.

Source Code:

```
import java.util.Random;

public class OddOnesMatrix {
    public static void main(String[] args) {
        int[][] matrix = new int[6][6];
        Random rand = new Random();

        // Fill matrix with 0s and 1s
        for (int i = 0; i < 6; i++) {
            for (int j = 0; j < 6; j++) {
                matrix[i][j] = rand.nextInt(2);
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

```

    }

    // Check rows
    for (int i = 0; i < 6; i++) {
        int count = 0;
        for (int j = 0; j < 6; j++) {
            if (matrix[i][j] == 1) count++;
        }
        if (count % 2 == 0) {
            System.out.println("Row " + i + " does not have an odd number of 1s.");
        }
    }

    // Check columns
    for (int j = 0; j < 6; j++) {
        int count = 0;
        for (int i = 0; i < 6; i++) {
            if (matrix[i][j] == 1) count++;
        }
        if (count % 2 == 0) {
            System.out.println("Column " + j + " does not have an odd number of 1s.");
        }
    }
}
}

```

Output:

```

0 1 0 0 1 0
0 0 0 0 1 0
0 0 1 1 0 0
0 1 1 0 0 1
1 1 1 1 1 1
0 0 0 1 1 0
Row 0 does not have an odd number of 1s.
Row 2 does not have an odd number of 1s.
Row 4 does not have an odd number of 1s.
Row 5 does not have an odd number of 1s.
Column 4 does not have an odd number of 1s.
Column 5 does not have an odd number of 1s.

```

Conclusion: This program provides an excellent exercise in working with 2D arrays, understanding array indexing, and applying logic-based validation using loops and conditionals. It also teaches how to implement random binary data generation, matrix display, and parity checking (odd/even counting) — essential skills in areas like image processing, error detection, and grid-based simulations.

Question-Answers:

1. What is a multidimensional array in Java?

A multidimensional array is an array of arrays. The most commonly used is the two-dimensional (2D) array, which stores elements in a row and column structure like a matrix.

2. What is a nested loop?

A nested loop is a loop inside another loop. In the context of 2D arrays, the outer loop typically iterates over rows, and the inner loop iterates over columns.

3. Why is a nested loop needed for 2D arrays?

Because each element of a 2D array is accessed using two indices — one for rows and one for columns — a single loop isn't enough to cover both dimensions.

4. Can you use enhanced for-each loops for 2D arrays?

Yes, but for full control over indices (like counting 1s), regular for loops are preferred.

Practical No: 12

Aim: Write a program that creates a Random object with seed 1000 and displays the first 100 random integers between 1 and 49 using the NextInt (49) method.

Theory:

This practical demonstrates use of the Random class, looping constructs, and seeding a random generator for reproducibility.

Java provides the Random class for generating pseudo-random numbers. These are not truly random but generated using an internal mathematical algorithm that gives a sequence of values that appear random.

A Random object can then be created with or without a seed:

```
Random rand = new Random(); // without seed
```

```
Random rand = new Random(1000); // with seed
```

The purpose of seeding is to make the random number sequence **reproducible**. If a seed is provided, the sequence of numbers generated will always be the same, which is useful for testing, debugging, and simulations.

A loop is required to generate **100 random numbers**. A for loop is most suitable here because the number of iterations is known in advance

This loop generates and prints each random number on the same line.

Source Code:

```
import java.util.Random;

public class SeededRandom {
    public static void main(String[] args) {
        Random rand = new Random(1000);
        for (int i = 0; i < 100; i++) {
            int num = rand.nextInt(49) + 1;
            System.out.print(num + " ");
        }
    }
}
```

Expected Output:

```
41 2 18 1 47 47 5 34 10 41 26 14 18 30 31 32 43 36 49 41 12 14 11 1 39 10 1 11 36 11
15 27 35 36 32 44 48 36 3 34 17 49 46 44 6 30 2 36 1 26 29 43 26 3 34 31 19 28 5 29 32
36 10 14 34 13 19 37 40 8 18 32 22 27 48 40 12 41 12 27 49 27 28 33 20 31 27 5 8 41 10
42 9 38 4 35 11 37 5 22
```

Conclusion:

This program strengthens the understanding of random number generation, how to control randomness using seeds, how to generate numbers in a specific range, the integration of object-oriented programming with control structures. It also reinforces how randomization can be predictable and controlled when required, which is critical in unit testing, game development, and simulations.

Question-Answers:

1. What is the Random class in Java?

The Random class, part of the java.util package, is used to generate pseudo-random numbers of different data types such as int, float, double, boolean, etc.

2. What is the Random class in Java used for?

The Random class in Java is used to generate pseudo-random numbers such as integers, floats, doubles, booleans, and more.

3. What is a seed in random number generation?

A seed is an initial value that determines the starting point of the random number sequence. Using the same seed will always generate the same sequence of random numbers.

4. Why is seeding useful in Java programs?

Seeding is useful for reproducibility — when testing or debugging programs that rely on random data, using a fixed seed ensures consistent results across runs.

5. What is the purpose of using seeding in testing?

To ensure that the test results are predictable and repeatable by generating the same sequence of random values every time.

Practical No: 13

Aim: Write a program to implement Inheritance & its types

Theory: This practical introduces inheritance, superclass and subclass relationships, and the use of the extends keyword.

In Java, inheritance is one of the core concepts of Object-Oriented Programming (OOP). It allows one class (called the subclass or child class) to acquire the properties (fields/variables) and behaviors (methods) of another class (called the superclass or parent class). This concept is used to promote code reuse, modularity, and extensibility, making programs easier to manage and scale.

When a class inherits from another, it can access the non-private fields and methods of the superclass. It can also override or extend them, depending on the requirements. This mechanism also supports method overriding, where a subclass provides a specific implementation of a method that is already defined in its superclass.

Source Code:

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output:

```
This animal eats food.
The dog barks.
```

Conclusion:

This practical demonstrates how code extensibility and reduction in redundancy can be achieved through inheritance, which is essential for building scalable and maintainable Java applications

Question-Answers:

1. What is inheritance in Java?

Inheritance is a mechanism in Java where one class (child class) inherits the properties and methods of another class (parent class), enabling code reusability and method overriding.

2. What keyword is used for inheritance?

The extends keyword is used to define inheritance between classes.

3. What are the benefits of inheritance?

Code reuse, Method overriding, Organized and modular code & Establishment of IS-A relationships

4. Can a subclass access private members of a superclass?

No, private members of a superclass are not accessible directly by the subclass.

Practical No: 14

Aim: Write a program to implement Exception handling

Theory:

In Java, exception handling is a mechanism used to handle runtime errors, ensuring the normal flow of the application is maintained. When an error occurs during the execution of a program, it may disrupt the normal flow and cause the program to crash. Exception handling provides a way to gracefully deal with such errors using try-catch blocks.

An **exception** is an event that occurs during the execution of a program that disrupts the normal flow of instructions. It is an **object** which is thrown at runtime and can be caught and handled.

Java has two main categories of exceptions:

1. **Checked Exceptions** – Checked at compile time (e.g., IOException, SQLException).
2. **Unchecked Exceptions** – Checked at runtime (e.g., ArithmeticException, NullPointerException).

Source Code:

```
public class ExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Division by zero is not allowed.");  
        }  
    }  
}
```

Output:

Error: Division by zero is not allowed.

Conclusion:

By implementing exception handling in Java, we learn how to write programs that can **gracefully manage runtime errors** using try, catch, and optionally finally blocks.

This promotes better **error control**, ensures the **stability** of the application, and enhances the overall **user experience** by preventing unexpected crashes.

It also lays the foundation for building **robust, secure, and maintainable Java applications**.

Question-Answers:

1. What is an exception in Java?

An exception is an event that disrupts the normal flow of a program during execution. It is an object that describes an error condition.

2. What is exception handling in Java?

Exception handling is a mechanism to handle runtime errors so that the normal flow of the application can be maintained without crashing.

3. What is the purpose of a try & catch block?

The try block contains code that might throw an exception during execution.

The `catch` block handles the exception thrown by the try block and prevents program termination.

4. What is a finally block?

The finally block contains code that always executes, regardless of whether an exception is thrown or caught, typically used for cleanup.

5. Can we have multiple catch blocks for a single try block?

Yes, you can use multiple catch blocks to handle different types of exceptions individually.

6. What happens if an exception is not handled?

If an exception is not caught or handled, the program will terminate and the JVM will print a stack trace.

7. What is the use of the throw keyword?

The throw keyword is used to manually throw an exception from a method or block

Practical No: 15

Aim: Write a Program to implement multithreaded program

Theory:

Multithreading is a programming technique that enables concurrent execution of two or more parts (threads) of a program to improve performance, especially in programs involving I/O, animations, or parallel processing. Java provides built-in support for multithreaded programming through the Thread class and the Runnable interface.

Java supports two main ways to create and start a thread:

1. By Extending the Thread Class
2. By Implementing the Runnable Interface

Both approaches require overriding the run() method which contains the code to be executed in the thread. To begin execution, the start() method is called, which invokes run() in a new call stack.

Source Code:

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

Output:

```
1
2
3
4
5
```

Conclusion:

This program teaches how to create and start multiple threads in Java, understand thread lifecycle and execution model, use both Thread class and Runnable interface, apply concurrency to solve real-world problems.

Question-Answers:

1. What is a thread in Java? What is multithreading?

A thread is a lightweight subprocess, the smallest unit of execution. It allows concurrent execution of two or more parts of a program.

Multithreading is a Java feature that allows the execution of two or more threads simultaneously for better CPU utilization and performance.

2. Which method starts a thread?

The start() method is used to begin execution of a thread, which internally calls the run() method.

3. What is the role of the run() method?

The run() method contains the code that is executed by the thread when it runs.

4. What is the lifecycle of a thread in Java?

New, Runnable, Running, Blocked/Waiting & Terminated (Dead)

5. Can we call the run() method directly instead of start()?

Yes, but it will not run the thread in a new call stack; it will behave like a normal method call.

6. What is the use of sleep() method in threading?

Thread.sleep(milliseconds) pauses the execution of the current thread for a specified time.

7. Why is multithreading important?

It improves application performance, responsiveness, and CPU efficiency, especially in tasks like real-time processing, animations, or parallel I/O operations.

Practical No: 16

Aim: Write a GUI program that use button to move the message to the left and right and use the radio button to change the colour for the message displayed

Theory:

GUI applications are event-driven programs that respond to user input through graphical components.

JavaFX is a modern GUI (Graphical User Interface) framework for Java, used to build rich internet applications and desktop applications. It is the successor to Swing and provides a more flexible, feature-rich API.

Features:

1. Built-in support for **2D and 3D graphics**.
2. Supports **CSS styling** and **FXML (XML-based UI layout)**.
3. Provides **multimedia support** (video and audio).
4. Includes a rich set of **UI controls** (Button, Label, TextField, etc.).
5. Uses a **scene graph** to build UI components

Source Code:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class MoveTextApp extends Application {
    private Text message = new Text(50, 50, "Java is Fun!");

    public void start(Stage stage) {
        Button left = new Button("Left");
        Button right = new Button("Right");
        RadioButton red = new RadioButton("Red");
        RadioButton blue = new RadioButton("Blue");

        ToggleGroup tg = new ToggleGroup();
        red.setToggleGroup(tg);
        blue.setToggleGroup(tg);

        HBox controls = new HBox(10, left, right, red, blue);
        Pane pane = new Pane(message);

        BorderPane root = new BorderPane();
        root.setCenter(pane);
        root.setBottom(controls);

        left.setOnAction(e -> message.setX(message.getX() - 10));
```

```

        right.setOnAction(e -> message.setX(message.getX() + 10));
        red.setOnAction(e -> message.setFill(Color.RED));
        blue.setOnAction(e -> message.setFill(Color.BLUE));

        Scene scene = new Scene(root, 400, 200);
        stage.setTitle("Move Message");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Output:

Conclusion: The JavaFX program allows user interaction with buttons and radio buttons.

Question-Answers:

1. What is JavaFX
JavaFX is a modern GUI toolkit for Java that provides features like 2D/3D graphics, CSS styling, and multimedia support
2. **What are the main components of a JavaFX application?**
Stage – Top-level container or window.
Scene – Holds UI components.
Nodes – UI controls or shapes like Button, Label, etc.
Application – The base class for launching the app.
3. **How is styling done in JavaFX?**
JavaFX supports CSS (Cascading Style Sheets) for styling UI components, similar to styling in web development.
4. **List some commonly used JavaFX UI controls.**
Label, Button, TextField, CheckBox, RadioButton, TextArea, ComboBox, ListView

Practical No: 17

Aim: Write a Program to implement Class Modeling

Theory: This program models real-world objects using classes and objects, showcasing data members and methods. Class modeling reflects object-oriented design where entities are represented as classes with attributes and behaviors.

Core concept in class modelling are: Class, Object Attributes & methods.

Class model provides the relation between the classes: following are the relations in class modelling:

3. Association: A "uses-a" relationship between classes.
4. Aggregation: A "has-a" relationship; a class contains another class but can exist independently.
5. Inheritance: A stronger form of aggregation; the contained object cannot exist without the container.
6. Polymorphism: An "is-a" relationship; subclass inherits from a superclass.

Importance of class modelling:

1. Helps in **visualizing** the structure of software.
2. Encourages **reusability** and **modularity**.
3. Makes the system easier to **maintain and extend**.
4. Improves **team communication** through visual diagrams.

Source Code:

```
class Student {
    String name;
    int rollNo;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

public class StudentModel {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Diksha";
        s.rollNo = 101;
        s.display();
    }
}
```

Output:

```
Name: Diksha
Roll No: 101
```

Conclusion: In this practical, we successfully explored the concept of class modeling using Java. By designing classes with attributes, methods, and relationships (such as association, inheritance, and aggregation), we understood how real-world entities can be modeled effectively in object-oriented programming.

Question-Answers:

1. What is class modeling in Java?

Class modeling in Java is the process of designing and representing classes and their relationships in a software system. It defines how real-world entities are mapped to software objects using classes, attributes, and methods. It helps organize the code using object-oriented principles like abstraction, encapsulation, inheritance, and polymorphism.

2. What are the main components of a class in Java?

The main components of a class in Java are:

- **Fields (Attributes):** Represent data or state of an object.
- **Methods (Functions):** Represent behavior or actions of the object.
- **Constructors:** Special methods used to initialize objects.
- **Access Modifiers:** Control access level (public, private, protected).
- **Inner Classes (optional):** Classes defined within another class.

3. Explain the types of relationships in class modeling.

1. **Association:** General relationship between two classes (e.g., a teacher uses a projector).
2. **Aggregation:** "Has-a" relationship; the contained object can exist independently (e.g., a department has teachers).
3. **Composition:** Stronger "has-a"; the contained object cannot exist without the container (e.g., a house has rooms).
4. **Inheritance:** "Is-a" relationship; a class inherits from another (e.g., a student is a person).
5. **Dependency:** One class depends temporarily on another to perform a task.

4. What is the significance of class diagrams in class modeling?

Class diagrams are a visual tool used in UML to represent the structure of a system. They show:

- Classes with attributes and methods.
 - Relationships (association, inheritance, etc.)
 - Visibility of members (+ for public, - for private).
- They help developers plan, understand, and maintain the system more easily.